LERO Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

CS5741
Concurrency & Parallelism in Software Development

Lecture 4 – Common Faults in Concurrent & Parallel Programs

Dr Faeq Alrimawi

HOST INSTITUTION
UNIVERSITY OF LIMERICK OLLSCOIL LUIMNIGH

PARTNER INSTITUTIONS
NUI Galway OE Gaillimh · Maynooth University · Trinity College Dublin Coláiste na Tríonóide, Baile Átha Cliath The University of Dublin · University College Dublin An Coláiste Ollscoile, Baile Átha Cliath · UCC · MTU Ollscoil Teicneolaíochta na Mumhan Munster Technological University · DCU · Waterford Institute of Technology · Dundalk Institute of Technology Dhún Dealgan · LIT · GMIT

Recap... Parallel Algorithms

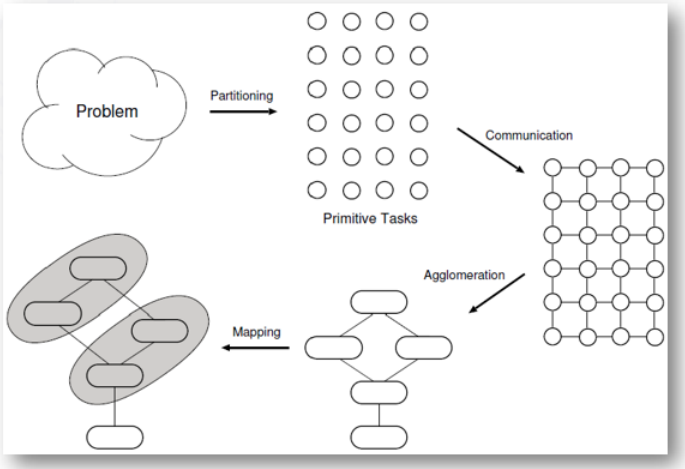
12

Key Concepts

Task Decomposition	Task Dependencies	Load Balancing	Synchronisation
• Dividing the problem into smaller tasks that can be executed concurrently. • Types: Data Decomposition (splitting data) and Functional Decomposition (splitting functions or tasks).	• Understanding dependencies between tasks to avoid incorrect execution. • Dependencies: Data dependencies, control dependencies, and resource dependencies.	• Evenly distributing work across processors to avoid idle time. • Strategies: Static (predefined distribution) vs. Dynamic (load balancing at runtime).	• Ensuring that tasks operate correctly when they need to access shared resources. • Mechanisms: Locks, barriers, and atomic operations.

26

Foster's Design Methodology (FDM)



Today, We Are Covering...

- Types of Faults
- Race conditions
- Deadlocks
- Livelock
- Starvation
- Best Practices for Avoiding Faults

Types of Faults

Race Conditions

- Happens when two or more tasks **access shared resources** (e.g., memory, variables) at the same time, and the final result depends on the **non-deterministic order** in which these accesses happen.

Deadlocks

- Happens when two or more **tasks are blocked forever**, each waiting for the other to release resources.

Livelocks

- Happens when tasks **continuously change** their **states** in response to each other **without** making **progress.**

Starvation

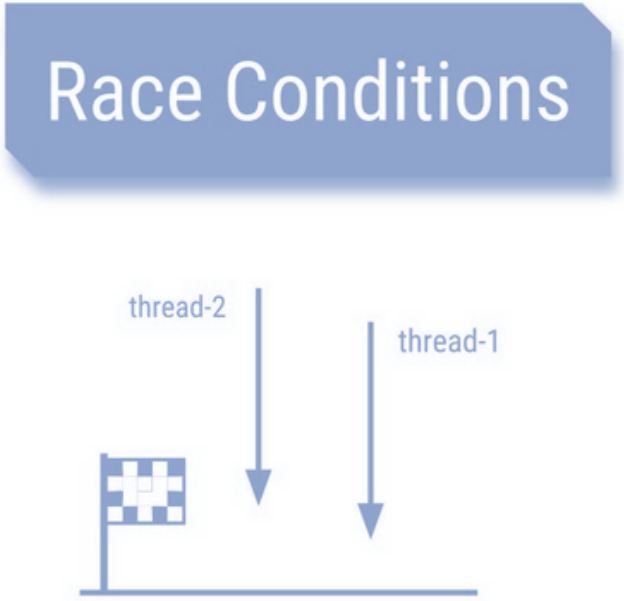
- Happens when a thread or process is **continuously denied access to resources** it needs to make progress, often because other threads monopolise those resources.

Today, We Are Covering...

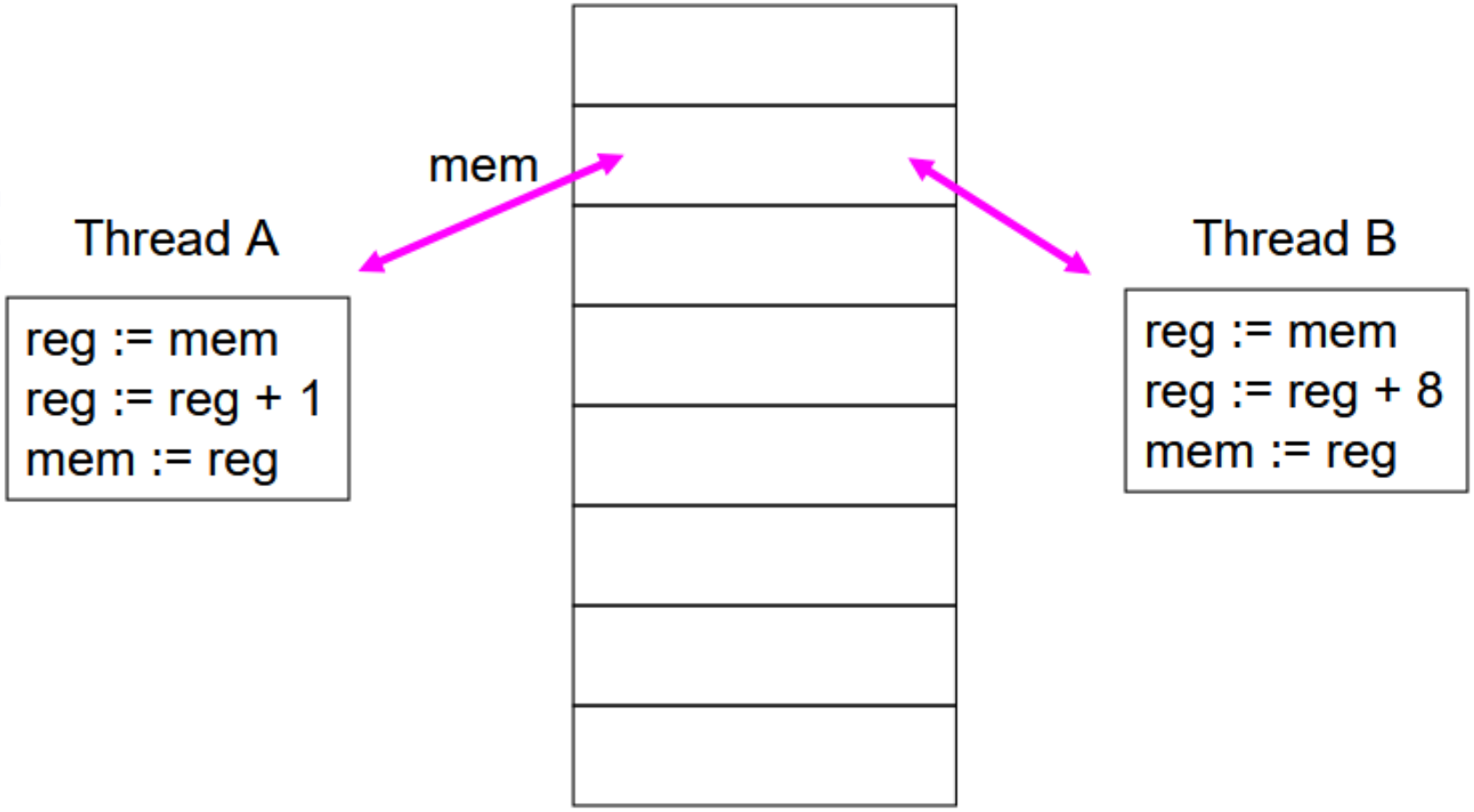
- Types of Faults
- Race conditions
- Deadlocks
- Livelock
- Starvation
- Best Practices for Avoiding Faults

Race Conditions

- Two or more tasks **access shared resources** (e.g., memory, variables) at the same time, and the final result depends on the **non-deterministic order** in which these accesses happen.
- A task may **read or write data** while **another** task is **modifying it**, leading to unpredictable or incorrect results.
- The **outcome** of the program **depends on** the **relative timing** of task execution, which can vary from one run to another.



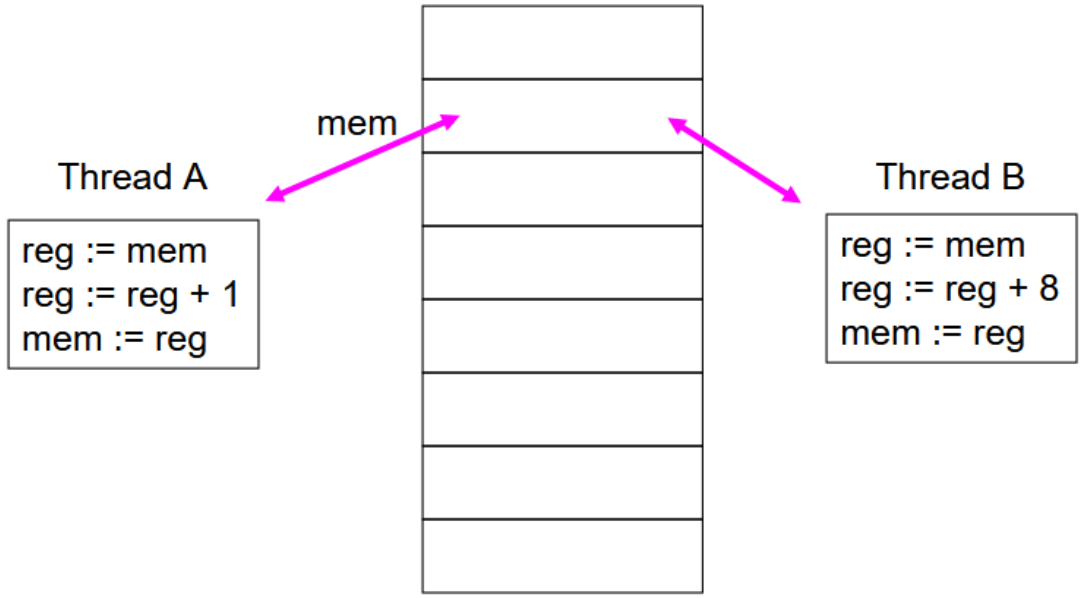
Race Conditions



Race Conditions – Scenario 1

Thread A: Read ➡ Modify ➡ Write

Thread B: Read ➡ Modify ➡ Write



Time	reg of A	reg of B	mem
1			0
2	0 (reg:=mem)	0 (reg:=mem)	
3	1 (reg:= reg+1)	8 (reg:= reg+8)	
4			?
5			
6			
7			

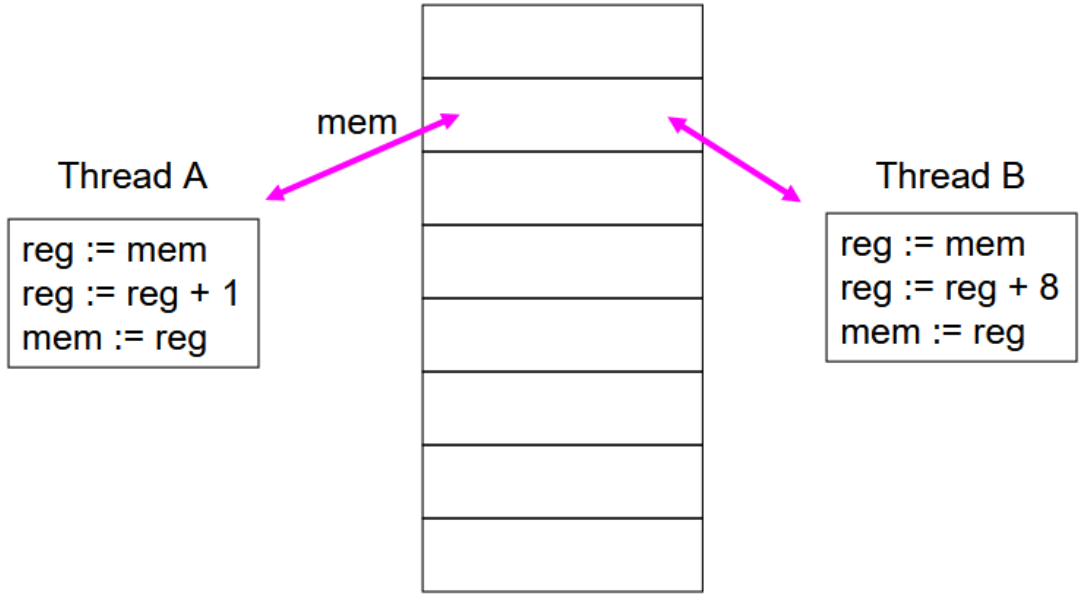
reg := mem
reg := reg + n
mem := reg

The race to write to mem creates a wrong result.

Race Conditions – Scenario 2

Thread A: Read ➡ Modify ➡ Write

Thread B: Read ➡ Modify ➡ Write



Time	reg of A	reg of B	mem
1			0
2	0 (reg:=mem)		0
3	1 (reg:= reg+1)	0 (reg:=mem)	
4		8 (reg:= reg+8)	1 (mem:=A.reg)
5			8 (mem:=B.reg)
6			
7			

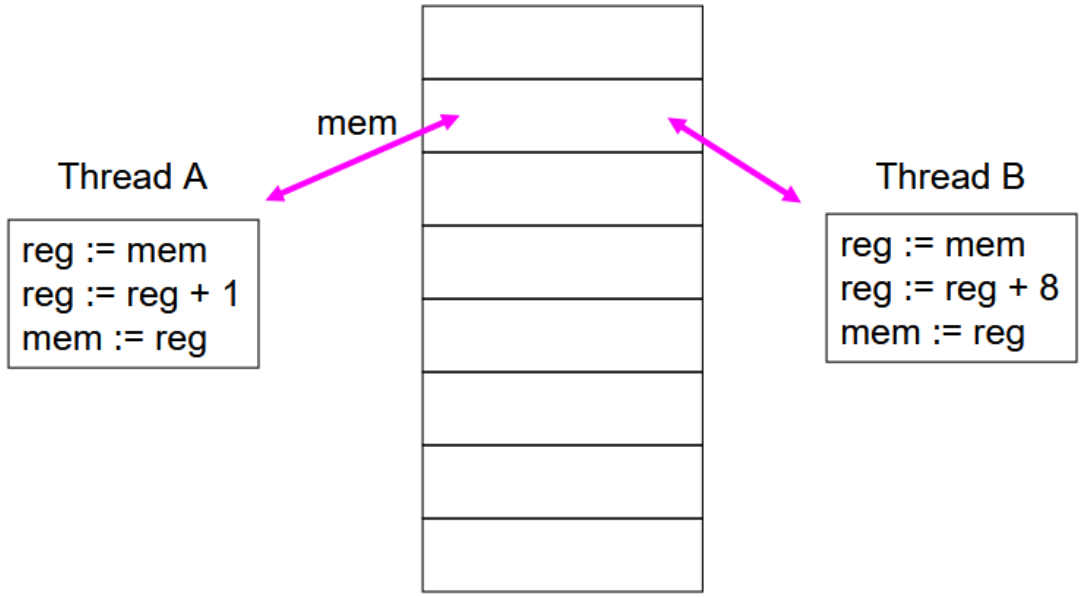
reg := mem
reg := reg + n
mem := reg

The update from process A is lost because it was not seen by B

Race Conditions – Scenario 3

Thread A: Read ➡ Modify ➡ Write

Thread B: Read ➡ Modify ➡ Write



Time	reg of A	reg of B	mem
1			0
2	0 (reg:=mem)		
3	1 (reg:= reg+1)		
4			1 (mem:=A.reg)
5		1 (reg:=mem)	
6		9 (reg:= reg+8)	
7			9 (mem:=B.reg)

```
reg := mem
reg := reg + n
mem := reg
```

The two updates do not interact, result is as expected.
By sheer luck, operations were *serialized*.

Code Example

- **Bank Account Withdrawal:** two threads attempt to withdraw money from the same bank account at the same time.

Thread A
withdraw(600)

Thread B
withdraw(500)

```
public void withdraw(int amount) {

    if(amount > balance) {
        System.err.println(x:"Error: Withdrawl amount exceeds current balance");
    } else {

        int newBalance = balance - amount;
        balance = newBalance;
    }
}
```

Initial balance = 1000

What can go wrong!?

Avoiding Race Conditions



Synchronisation Mechanisms

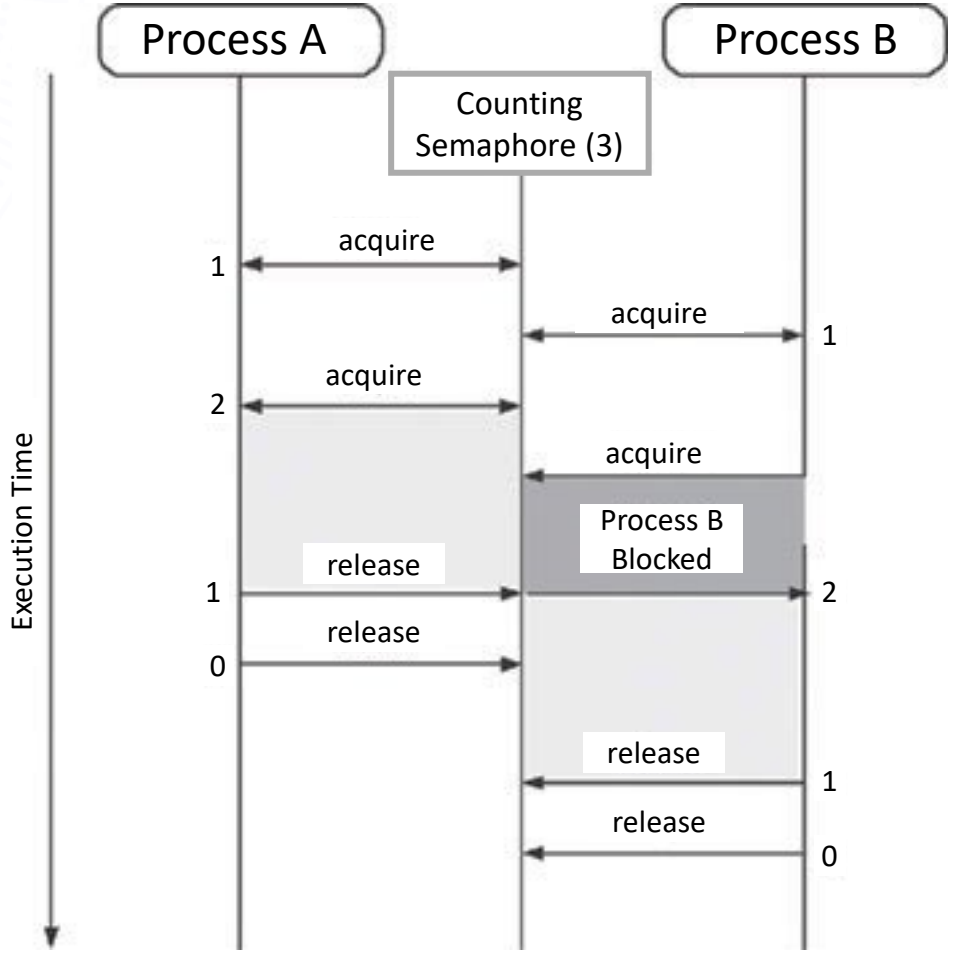
- Semaphore
  - Guards N resources.
- Mutex
  - Guards ONE resource. (binary semaphore)
- Monitor
  - A mutex with a Thread queue. (allows wait and notify)

Semaphore

- Maintains a **counter** that tracks the number of permits available to access the resource.
  - Limits the number of threads that can access the resource concurrently.

- Two Types:
  - **Binary Semaphore**: works **like a mutex** with only two states (0 and 1), allowing either full access or no access.
  - **Counting Semaphore**: Allows a **limited number** of **threads** to access a resource concurrently.
    - The counter decrements when a thread enters and increments when a thread leaves.

Counting Semaphore



Counting Semaphore: Java Example

```java
public class SemaphoreExample {
    Run | Debug
    public static void main(String[] args) {
        // Only five threads can access the resource at a time. Any more threads have to wait
        Semaphore semaphore = new Semaphore(5);
        SharedResource resource = new SharedResource();

        Runnable task = () -> {
            try {
                semaphore.acquire();
                resource.accessResource();
                Thread.sleep(millis:100); // Simulate some work
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                semaphore.release();
            }
        };

        Thread t1 = new Thread(task, name:"Thread 1");
        Thread t2 = new Thread(task, name:"Thread 2");

        t1.start();
        t2.start();
    }
}
```

Use Cases of Counting Semaphore

- **Connection pools** (e.g., database connection pools): limit the number of concurrent connections to a database.
- **Thread pools**: control how many threads can execute certain tasks simultaneously.
- **Resource allocation**: managing limited resources like **printers** or network ports.

Mutex

- Allows **only one thread** to own the lock at a time.
- Low level structure; developers have to lock and release.

- Key Characteristics
  - **Ownership**: a mutex is **owned by the thread that locks it**, and only the owning thread can unlock it.
  - **Blocking**: If a thread tries to acquire a locked mutex, it will **block** (wait) **until** the mutex is **released**.
  - **Simple Usage**: Mutexes provide a straightforward way to ensure mutual exclusion (only one thread can enter a critical section at a time).

Mutex: Java Example

```java
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class MutexExample {
    private final Lock lock = new ReentrantLock();

    public void criticalSection() {
        lock.lock();
        try {
            // Critical section of the code
            System.out.println("Thread " + Thread.currentThread().getName() + " is in the critical section.");
        } finally {
            lock.unlock(); // Always release the lock
        }
    }
}
```

Monitor

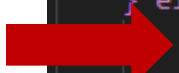
- A high-level synchronisation construct that **combines mutexes and condition variables** (wait and signal) to simplify thread coordination.
  - Lock and release are controlled internally.

- In **Java**, the keyword ***synchronized*** is used.

Monitor: Example



```java
public synchronized void withdraw(int amount) {
    if (amount > balance) {
        System.out.println(Thread.currentThread().getName() + " insufficient funds to withdraw " + amount);
    } else {
        balance = balance - amount;
        System.out.println(Thread.currentThread().getName() + " completed withdrawal. Remaining balance: " + balance);
    }
}
```



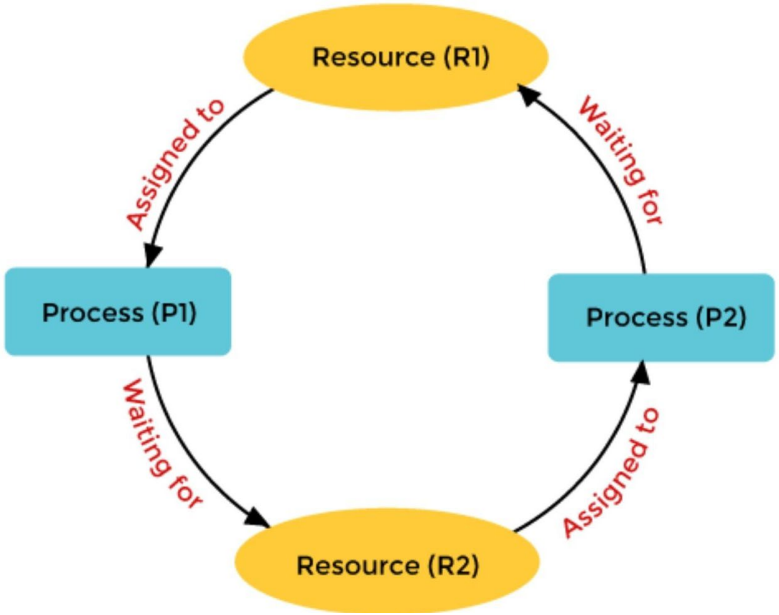
```java
public void withdraw(int amount) {
    if (amount > balance) {
        System.out.println(Thread.currentThread().getName() + " insufficient funds to withdraw " + amount);
    } else {
        synchronized (this) {
        balance = balance - amount;
        }
        System.out.println(Thread.currentThread().getName() + " completed withdrawl. Remaining balance: " + balance);
    }
}
```

Today, We Are Covering…

- Types of Faults
- Race conditions
- **Deadlocks**
- Livelock
- Starvation
- Best Practices for Avoiding Faults

Deadlocks

- Two or more processes or threads are **unable to proceed** because each is **waiting** for the other to **release** a **resource** that they need.
  - They are 'stuck!'
- Four conditions for a deadlock to happen
  - **Mutual Exclusion**: at least one **resource held** in a **non-shareable mode**.
    - E.g., two processes trying to access one printer that allows only one process at a time.
  - **Hold and Wait**: a process **holding** one **resource** is **waiting** on **another** resource.
    - E.g., a process holding a file and waiting to access a database that another process is holding.
  - **No Preemption**: resources cannot be forcibly taken away from a process
    - One process holding a lock on a file, another process cannot force it to release it.
  - **Circular Wait**: a circular chain of processes, where one process is waiting for a resource from another in the chain.
    - *Process A* holds *Resource 1* and is waiting for *Resource 2*, while *Process B* holds *Resource 2* and is waiting for *Resource 1*.



Deadlock Example

- Thread 1 holds resource1 and waits for resource2.
- Thread 2 holds resource2 and waits for resource1.
- Both threads are stuck, waiting for the other to release its resource, causing a deadlock.

```java
public class DeadlockExample {
    // Shared resources
    private final Object resource1 = new Object();
    private final Object resource2 = new Object();

    public void method1() {
        synchronized (resource1) {
            System.out.println(x:"Thread 1: Holding resource 1...");

            try { Thread.sleep(millis:100); } catch (InterruptedException e) {}

            System.out.println(x:"Thread 1: Waiting for resource 2...");
            synchronized (resource2) {
                System.out.println(x:"Thread 1: Acquired resource 2!");
            }
        }
    }

    public void method2() {
        synchronized (resource2) {
            System.out.println(x:"Thread 2: Holding resource 2...");

            try { Thread.sleep(millis:100); } catch (InterruptedException e) {}

            System.out.println(x:"Thread 2: Waiting for resource 1...");
            synchronized (resource1) {
                System.out.println(x:"Thread 2: Acquired resource 1!");
            }
        }
    }
```

Deadlock Prevention Techniques

To prevent deadlocks, one needs to **break at least one of the four conditions**

- Eliminate Mutual Exclusion, if possible
- Eliminate Hold and Wait
  - Processes should ask for permission to access ALL resources from the start
- Eliminate No Preemption
  - A process should release resources if denied access to others
- Eliminate Circular Wait
  - Impose an ordering on the resources and force processes to request resources in a predefined order

Deadlock Detection & Recovery

- Detection
  - Some systems allow deadlocks to occur but include a mechanism to detect when deadlocks have happened.
  - Once a deadlock is detected, the system takes action to recover, such as terminating a process involved in the deadlock or forcibly releasing a resource.

- Recovery Techniques
  - **Abort Processes**: Terminate one or more processes involved in the deadlock to break the circular wait.
  - **Resource Preemption**: Temporarily take away resources from other processes and reassign them to the process in deadlock.

Today, We Are Covering...

- Types of Faults
- Race conditions
- Deadlocks
- Livelock
- Starvation
- Best Practices for Avoiding Faults

Livelocks

- It is when two or more threads/processes **continuously change their state** in response to each other, but **no progress** is made.

- Makes them **harder to detect** than deadlocks.
  - Threads/processes APPEAR to be doing something!

Livelocks: Example

```java
class LivelockExample {

    static class Worker {
        private String name;
        private boolean active = true;

        Worker(String name) {
            this.name = name;
        }

        public boolean isActive() {
            return active;
        }

        public void doWork(Worker other) {

            while (active) {

                // "Be polite" and let the other worker go first
                if (other.isActive()) {
                    System.out.println(
                        name + ": You go first, " + other.name
                    );

                    // Keep retrying
                    Thread.yield();
                    continue;
                }

                System.out.println(name + ": I am working!");

                active = false;
            }
        }
    }

    public static void main(String[] args) {

        Worker alice = new Worker(name: "Alice");
        Worker bob = new Worker(name: "Bob");

        Thread t1 = new Thread(() -> alice.doWork(bob));
        Thread t2 = new Thread(() -> bob.doWork(alice));

        t1.start();
        t2.start();
    }
}
```

```
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
Bob: You go first, Alice
```

Avoiding Livelocks

- Timeout Mechanisms
  - Use a timeout or a limit for tries.
- Randomised Backoff
  - Instead of a fixed retry delay, use a random retry delay.
- Give Priority to One Thread
  - Assign higher priority to a thread over the other.

Today, We Are Covering…

- Types of Faults
- Race conditions
- Deadlocks
- Livelock
- Starvation
- Best Practices for Avoiding Faults

Starvation

- It is when a thread/process is **constantly denied access to resources** it needs to proceed.
  - Resources are available, but constantly assigned to other threads.
- Happens when some threads to monopolise resources.
  - Greedy threads!

Starvation: Example

```java
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class StarvationExample {
    // Fairness flag set to true
    private static Lock lock = new ReentrantLock(fair:false);

    Run | Debug
    public static void main(String[] args) {
        // Creating three threads with varying priorities
        Thread greedyPriority = new Thread(new Task(),
        name:"Greedy Thread");
        Thread normalPriority = new Thread(new Task(),
        name:"Normal Thread 1");

        // Starting threads
        greedyPriority.start();
        normalPriority.start();
    }
```

Change it to true

1

```java
    static class Task implements Runnable {
        @Override
        public void run() {
            for (int i = 0; i < 5; i++) {
                try {
                    lock.lock();
                    System.out.println(Thread.currentThread().getName()
                    + " acquired the lock and is working.");
                    // Simulate some work with the resource
                    Thread.sleep(millis:100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } finally {
                    System.out.println(Thread.currentThread().getName()
                    + " released the lock.");
                    lock.unlock();
                }
            }
        }
    }
}
```

2

Strategies to Prevent Starvation

- Fair Locking mechanisms
  - E.g., the ReentrantLock in java provides a fairness parameter.
- Aging in Scheduling
  - **Aging** gradually **increases** the **priority** of a waiting task over time.
- Avoiding Long Critical Sections
  - Break your critical sections into different parts to prevent a thread from locking the whole section and cause starvation.

Today, We Are Covering...

- Types of Faults
- Race conditions
- Deadlocks
- Livelock
- Starvation
- Best Practices for Avoiding Faults

Best Practices for Avoiding Faults

- Use High-Level Abstractions
  - Avoid manual thread management
  - E.g., using ForkJoinPool framework in Java
- Avoid Shared Mutable data
  - Prefer **immutable** objects.
  - If mutable state is necessary, ensure that access to the state is **synchronised** properly.
- Use Proper Synchronisation
  - If sharing mutable state is unavoidable, proper synchronisation mechanisms (e.g., semaphores, or monitors) must be used.

Best Practices for Avoiding Faults

- Avoid Nested Locks to Prevent Deadlocks
  - Always acquire locks in the **same order** across threads.
  - Use **lock timeouts** where possible, to prevent indefinite waiting.
- Use Thread-Safe Data Structures
  - Standard data structures (e.g., ArrayList, HashMap) are **not thread-safe**.
  - Use **concurrent data structures**, such as ConcurrentHashMap, CopyOnWriteArrayList.
- Limit Global State
  - Use local variables.
  - e.g., **ThreadLocal** in Java, which ensures that each thread has its own instance of the variable

Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

What Are Your Questions?

HOST INSTITUTION

UNIVERSITY OF LIMERICK OLLSCOIL LUIMNIGH

PARTNER INSTITUTIONS

NUI Galway OÉ Gaillimh · Maynooth University · Trinity College Dublin, Coláiste na Tríonóide, Baile Átha Cliath, The University of Dublin · University College Dublin, An Coláiste Ollscoile, Baile Átha Cliath · UCC · MTU, Ollscoil Teicneolaíochta na Mumhan, Munster Technological University · DCU · Waterford Institute of Technology · Dundalk Institute of Technology, Dún Dealgan · LIT · GMIT